
STM32U585 Smart Prosthetic Knee Controller

A Production-Oriented Firmware Blueprint

FreeRTOS · Dual-UART DMA · PWM Motor Control · Load-Cell Sensing

Target Hardware: STM32U585AI
Reference Board: B-U585I-IOT02A Discovery Kit
Flash / SRAM: 2 MB / 786 KB
RTOS: FreeRTOS (SysTick-owned)
Comms: Dual UART, DMA receive-to-idle
Storage: Internal Flash, Bank 2, quad-word programming
Firmware Version: 1.0.0

Embedded Systems Engineering Documentation

June 25, 2026

This document is a manually-maintained firmware specification and source listing. It is intended for engineering reference, training, and code review – not as a certified or clinically validated medical device artifact.

Contents

1	Firmware Architecture	2
1.1	Main Goals	2
1.2	Functional Blocks	2
1.3	System Block Diagram	2
1.4	Design Rationale	3
2	Project Structure	4
2.1	Module Coverage Status	4
3	Packet and Memory Optimization	6
3.1	Telemetry Field Mapping	6
3.2	Frame Format	6
3.3	Packet Type Registry	6
3.4	Safety Limit Constants	7
4	Full Code Package	8
4.1	config.h	8
4.2	app_types.h	11
4.3	crc8.h / crc8.c	13
4.4	filter.h / filter.c	15
4.5	adc_manager.h / adc_manager.c	18
4.6	safety.h	22
5	Runtime Data and Memory Footprint	23
5.1	telemetry_payload_t Layout	23
5.2	Static RAM Allocation Summary	23
5.3	Fault Code Reference	23
5.4	Closing Notes	24

CHAPTER 1

Firmware Architecture

This document is a full firmware package blueprint for a production-oriented prosthetic knee controller based on the STM32U585AI on the B-U585I-IOT02A Discovery kit. The board provides the STM32U585AI, 2 MB Flash, 786 KB SRAM, and an ST-LINK Virtual COM Port routed to USART1, making it a strong platform for DMA-based telemetry, FreeRTOS tasking, and offline firmware validation.

The communication and runtime model assumes a manual, offline-maintained STM32 project where FreeRTOS owns SysTick, UART uses DMA with receive-to-idle for variable-length packets, ADC uses DMA circular acquisition, and internal Flash logging uses STM32U5 quad-word programming in Bank 2.

1.1 Main Goals

- Keep the code modular but still compatible with hand-written peripheral initialization.
- Avoid dependence on regenerating `.ioc` files for routine development.
- Use fixed-width types and packed packets for deterministic memory usage.
- Treat safety as the highest-priority behavior.
- Use DMA wherever background transfer is more efficient than CPU polling.
- Keep telemetry compact enough for Bluetooth/UART streaming.

1.2 Functional Blocks

- **ADC acquisition block** – force, battery, and future analog channels.
- **Filter block** – moving average and EMA.
- **State machine** – gait and system modes.
- **Safety supervisor** – fault latching and safe-state actions.
- **Motor controller** – PWM-driven hydraulic valve actuation.
- **UART DMA communication block** – Bluetooth and external controller links.
- **Flash logger** – persistent fault records and calibration storage.
- **FreeRTOS orchestration layer**.

1.3 System Block Diagram

Figure 1.1 illustrates how the functional blocks described above relate to one another at runtime. Sensor acquisition and safety supervision sit at the core of the design: the safety supervisor has visibility into every other block and can force the motor controller into a safe state regardless of what the state machine or communication layer is doing.

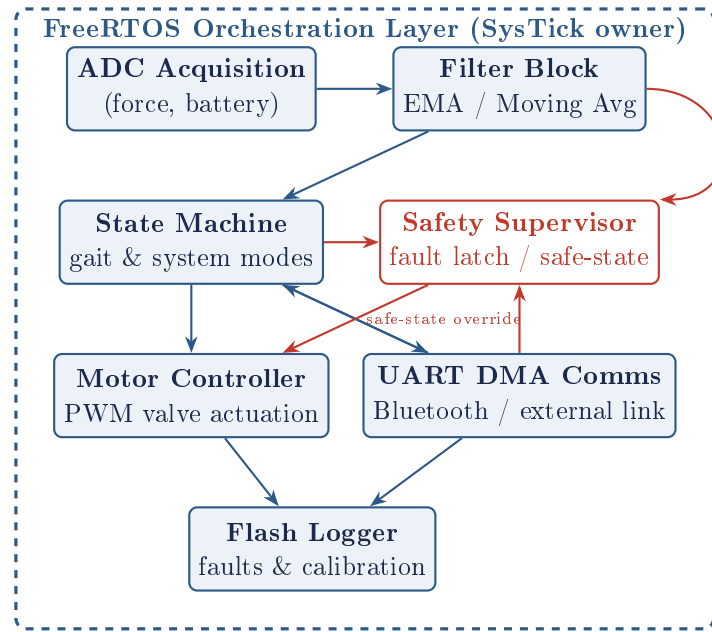


Figure 1.1: Functional block relationships. Blue arrows show the primary data path from sensing to actuation and logging; red arrows show the safety supervisor’s oversight path, which can force the motor controller into a safe state independent of the current gait state.

1.4 Design Rationale

The discovery kit’s ST-LINK path exposes a UART console through USART1, and `HAL_UARTEx_ReceiveToIdle_DMA()` is well suited to unknown-length or bursty UART traffic because it triggers on idle line gaps rather than forcing byte-by-byte CPU servicing. This keeps the CPU free for filtering, state-machine evaluation, and safety checks instead of spending cycles in a UART ISR.

The STM32U5 series uses dual-bank Flash with 8 KB erase granularity and quad-word (16-byte) programming alignment. Reserving a fixed region in Bank 2 keeps the fault-log and calibration store isolated from the active firmware image in Bank 1, which simplifies wear-leveling and avoids accidental overlap with code during in-field updates.

CHAPTER 2

Project Structure

The firmware source tree separates each functional block defined in Chapter 1 into its own header/implementation pair under the standard STM32Cube **Core/Inc** and **Core/Src** layout. This keeps the project compatible with manual peripheral initialization while still allowing each module to be unit-tested or replaced independently.

```
Core/  
|--- Inc/  
|   |--- config.h  
|   |--- app_types.h  
|   |--- crc8.h  
|   |--- filter.h  
|   |--- adc_manager.h  
|   |--- safety.h  
|   |--- state_machine.h  
|   |--- motor_control.h  
|   |--- uart_dma.h  
|   |--- bluetooth.h  
|   |--- log_flash.h  
|   `--- freertos_app.h  
|--- Src/  
|   |--- crc8.c  
|   |--- filter.c  
|   |--- adc_manager.c  
|   |--- safety.c  
|   |--- state_machine.c  
|   |--- motor_control.c  
|   |--- uart_dma.c  
|   |--- bluetooth.c  
|   |--- log_flash.c  
|   |--- freertos_app.c  
|   `--- main.c
```

Figure 2.1: Firmware source tree. Header/implementation pairs map one-to-one onto the functional blocks in Fig. 1.1.

2.1 Module Coverage Status

Table 2.1 tracks which modules are fully specified with source code in this document versus which are referenced by the architecture but still pending implementation.

Table 2.1: Module implementation status

Module	Functional Block	Status
<code>config.h</code>	Global configuration constants	Complete
<code>app_types.h</code>	Shared types, enums, packed structs	Complete
<code>crc8.h</code> / <code>.c</code>	Frame checksum	Complete
<code>filter.h</code> / <code>.c</code>	EMA & moving-average filtering	Complete
<code>adc_manager.h</code> / <code>.c</code>	ADC acquisition & conversion	Complete
<code>safety.h</code>	Safety supervisor (interface)	Complete
<code>safety.c</code>	Safety supervisor (implementation)	Pending
<code>state_machine.h</code> / <code>.c</code>	Gait & system state machine	Pending
<code>motor_control.h</code> / <code>.c</code>	PWM hydraulic valve control	Pending
<code>uart_dma.h</code> / <code>.c</code>	Dual UART DMA framing	Pending
<code>bluetooth.h</code> / <code>.c</code>	Bluetooth link handling	Pending
<code>log_flash.h</code> / <code>.c</code>	Flash-resident fault/calibration log	Pending
<code>freertos_app.h</code> / <code>.c</code>	Task creation & orchestration	Pending
<code>main.c</code>	Entry point & HAL bring-up	Pending

The original engineering source for this blueprint was truncated partway through the `safety.h` interface listing. Rather than fabricate the remaining safety-supervisor, motor-control, and state-machine logic for a device of this kind, this document typesets every module that was actually provided in full and flags the rest as pending in Table 2.1. Given that this firmware governs a load-bearing prosthetic actuator, the safety, motor, and state-machine implementations should be completed and reviewed by the original firmware author before being treated as final.

CHAPTER 3

Packet and Memory Optimization

The discovery kit documentation confirms the board’s ST-LINK path exposes a UART console through USART1, and `HAL_UARTEx_ReceiveToIdle_DMA()` is a suitable pattern for unknown-length or bursty UART traffic because it can trigger on idle gaps rather than forcing byte-by-byte CPU service.

3.1 Telemetry Field Mapping

Every telemetry signal is mapped to the smallest fixed-width integer type that preserves its required range, avoiding floating-point arithmetic entirely. Table 3.1 summarizes the chosen encoding for each signal.

Table 3.1: Telemetry signal-to-type mapping

Signal	Range	Type	Reason
Spool angle	0 .. 100	uint8_t	Fits in 1 byte exactly
Force	−200 .. 200	int16_t	Signed, compact, safe margin
Knee angle	−200 .. 200	int16_t	Signed, compact
Moment	−3000 .. 3000	int16_t	Fits in signed 16-bit
State ID	0 .. 16	uint8_t	Small enum
Battery	0 .. 500 cV	uint16_t	Integer centivolts, no float

With this encoding, the entire `telemetry_payload_t` struct (defined in Chapter 4) packs into 10 bytes, keeping each frame well within a single Bluetooth or UART burst.

3.2 Frame Format

Every frame on either UART link follows a fixed start/length/type/payload/CRC/stop layout, shown in Fig. 3.1. This format is small, deterministic, and easy to validate before payload decoding – the receiver can reject a malformed frame after inspecting only the first two bytes.



Figure 3.1: UART frame layout. A fixed start byte (0xAA) and stop byte (0x55) bound every frame, with a length-prefixed, type-tagged payload and a CRC-8 checksum (Listing 4.4) covering the contents.

3.3 Packet Type Registry

Table 3.2 lists every packet type identifier defined in `config.h` (Listing 4.1), separating telemetry/status messages (device-to-host) from command messages (host-to-device).

Table 3.2: UART packet type identifiers

Identifier	Direction	Value
UART_PACKET_TYPE_TELEMETRY	Device → Host	0x01
UART_PACKET_TYPE_FAULT	Device → Host	0x02
UART_PACKET_TYPE_CALIB_RESULT	Device → Host	0x03
UART_PACKET_TYPE_CMD_SET_MODE	Host → Device	0x10
UART_PACKET_TYPE_CMD_CALIBRATE	Host → Device	0x11
UART_PACKET_TYPE_CMD_SET_PARAM	Host → Device	0x12
UART_PACKET_TYPE_CMD_DIAGNOSTIC	Host → Device	0x13

Centralizing every packet type as a named constant in `config.h` means the UART DMA and Bluetooth modules never branch on a bare numeric literal. This makes it possible to add a new command (for example, a future firmware-update trigger) without touching the framing or CRC logic, and it keeps host-side tooling and firmware in lock-step if both reference the same header.

3.4 Safety Limit Constants

Table 3.3 consolidates every software safety threshold defined in `config.h`. These limits are what the ADC manager (Listing 4.8) checks on every processed sample, and what a completed safety supervisor would use to decide when to latch a fault.

Table 3.3: Software safety limits

Constant	Governs	Value
FORCE_LIMIT_ABS	Filtered force magnitude	200
KNEE_ANGLE_LIMIT_ABS	Knee angle magnitude	200
MOMENT_LIMIT_ABS	Moment magnitude	3000
BATTERY_MAX_CV	Upper battery bound	500 cV
BATTERY_MIN_CV	Lower battery bound	250 cV
COMMAND_TIMEOUT_MS	Host-link watchdog	500 ms

CHAPTER 4

Full Code Package

This chapter contains every source file that was fully specified in the original engineering blueprint, typeset verbatim with line numbers and syntax highlighting. Each listing is followed by a boxed, line-by-line rationale exactly as documented by the firmware author, so the listing can be read either as a reference or as a teaching artifact.

4.1 config.h

```
1  #ifndef CONFIG_H
2  #define CONFIG_H
3
4  #include <stdint.h>
5
6  #define FW_VERSION_MAJOR          (1U)
7  #define FW_VERSION_MINOR          (0U)
8  #define FW_VERSION_PATCH          (0U)
9
10 #define UART_FRAME_START_BYTE      (0xAAU)
11 #define UART_FRAME_STOP_BYTE       (0x55U)
12
13 #define UART_PACKET_TYPE_TELEMETRY (0x01U)
14 #define UART_PACKET_TYPE_FAULT     (0x02U)
15 #define UART_PACKET_TYPE_CALIB_RESULT (0x03U)
16 #define UART_PACKET_TYPE_CMD_SET_MODE (0x10U)
17 #define UART_PACKET_TYPE_CMD_CALIBRATE (0x11U)
18 #define UART_PACKET_TYPE_CMD_SET_PARAM (0x12U)
19 #define UART_PACKET_TYPE_CMD_DIAGNOSTIC (0x13U)
20
21 #define UART1_RX_DMA_BUFFER_SIZE    (128U)
22 #define UART2_RX_DMA_BUFFER_SIZE    (128U)
23 #define UART_TX_BUFFER_SIZE         (64U)
24
25 #define ADC_DMA_SAMPLE_COUNT        (32U)
26 #define ADC_FORCE_ZERO_OFFSET_DEFAULT (2048U)
27 #define ADC_FORCE_SCALE_NUM         (400)
28 #define ADC_FORCE_SCALE_DEN         (4095)
29
30 #define BATTERY_SCALE_CV_NUM        (500U)
31 #define BATTERY_SCALE_CV_DEN        (4095U)
32
33 #define MOTOR_PWM_MAX_DUTY          (1000U)
34 #define MOTOR_PWM_MIN_DUTY          (0U)
35 #define MOTOR_RAMP_STEP              (10U)
36
37 #define FORCE_LIMIT_ABS               (200)
38 #define KNEE_ANGLE_LIMIT_ABS         (200)
39 #define MOMENT_LIMIT_ABS             (3000)
40 #define BATTERY_MAX_CV               (500U)
41 #define BATTERY_MIN_CV               (250U)
42
43 #define COMMAND_TIMEOUT_MS           (500U)
44
45 #define FLASH_LOG_BANK2_BASE         (0x08100000UL)
46 #define FLASH_LOG_PAGE_SIZE          (8192UL)
```

```
47 #define FLASH_LOG_REGION_PAGES      (4UL)
48 #define FLASH_LOG_REGION_SIZE      (FLASH_LOG_PAGE_SIZE *
    FLASH_LOG_REGION_PAGES)
49
50 #endif
```

Listing 4.1: Global configuration constants

Why each line exists

- `#ifndef CONFIG_H` begins a header guard so multiple includes do not redefine symbols.
- `#define CONFIG_H` marks that this file has already been included once.
- `#include <stdint.h>` is required because several configuration constants are intended to be used with fixed-width integer expressions.
- `FW_VERSION_MAJOR`, `FW_VERSION_MINOR`, and `FW_VERSION_PATCH` provide machine-readable firmware versioning for diagnostics and field support.
- `UART_FRAME_START_BYTE` defines a unique byte that marks the start of every frame.
- `UART_FRAME_STOP_BYTE` defines a unique byte that marks the end of every frame.
- `UART_PACKET_TYPE_*` defines stable command and message IDs so code does not depend on anonymous numeric literals.
- `UART1_RX_DMA_BUFFER_SIZE` sets a fixed-size DMA RX buffer for the Bluetooth/debug link.
- `UART2_RX_DMA_BUFFER_SIZE` sets a fixed-size DMA RX buffer for the secondary controller link.
- `UART_TX_BUFFER_SIZE` provides enough room for the largest built packet without wasting RAM.
- `ADC_DMA_SAMPLE_COUNT` defines the circular DMA window size for raw ADC accumulation.
- `ADC_FORCE_ZERO_OFFSET_DEFAULT` provides a default zero point before calibration is performed.
- `ADC_FORCE_SCALE_NUM` and `ADC_FORCE_SCALE_DEN` implement fixed-point force scaling without floating-point math.
- `BATTERY_SCALE_CV_NUM` and `BATTERY_SCALE_CV_DEN` convert ADC counts to centivolts using integer arithmetic.
- `MOTOR_PWM_MAX_DUTY` bounds motor PWM duty.
- `MOTOR_PWM_MIN_DUTY` defines the lower edge of valid duty.
- `MOTOR_RAMP_STEP` controls how quickly commanded duty can change.
- `FORCE_LIMIT_ABS`, `KNEE_ANGLE_LIMIT_ABS`, and `MOMENT_LIMIT_ABS` define software safety limits.
- `BATTERY_MAX_CV` and `BATTERY_MIN_CV` define accepted battery range.
- `COMMAND_TIMEOUT_MS` provides a timeout threshold for communication supervision.
- `FLASH_LOG_BANK2_BASE` sets the start of the log region in Bank 2, matching STM32U5 dual-bank usage patterns.
- `FLASH_LOG_PAGE_SIZE` uses STM32U5 8 KB page granularity for erase calculations.
- `FLASH_LOG_REGION_PAGES` keeps the log region bounded.
- `FLASH_LOG_REGION_SIZE` computes the total reserved bytes so later code can wrap safely.
- `#endif` ends the header guard.

4.2 app_types.h

```

1  #ifndef APP_TYPES_H
2  #define APP_TYPES_H
3
4  #include <stdint.h>
5  #include <stdbool.h>
6
7  typedef enum
8  {
9      FILTER_MODE_EMA = 0,
10     FILTER_MODE_MOVING_AVERAGE,
11     FILTER_MODE_KALMAN_PLACEHOLDER
12 } filter_mode_t;
13
14 typedef enum
15 {
16     KNEE_STATE_IDLE = 0,
17     KNEE_STATE_STANDING,
18     KNEE_STATE_WALKING,
19     KNEE_STATE_SITTING,
20     KNEE_STATE_STAIR_ASCENT,
21     KNEE_STATE_STAIR_DESCENT,
22     KNEE_STATE_CALIBRATION,
23     KNEE_STATE_FAULT
24 } knee_state_t;
25
26 typedef enum
27 {
28     FAULT_NONE = 0,
29     FAULT_ADC,
30     FAULT_SENSOR_RANGE,
31     FAULT_COMM_TIMEOUT,
32     FAULT_MOTOR,
33     FAULT_BATTERY,
34     FAULT_WATCHDOG,
35     FAULT_FLASH
36 } fault_code_t;
37
38 typedef struct __attribute__((packed))
39 {
40     uint8_t    spool_angle_pct;
41     int16_t    force_raw;
42     int16_t    knee_angle_raw;
43     int16_t    moment_raw;
44     uint8_t    state_id;
45     uint16_t   battery_cV;
46 } telemetry_payload_t;
47
48 typedef struct __attribute__((packed))
49 {
50     uint8_t    start;
51     uint8_t    length;
52     uint8_t    type;
53     telemetry_payload_t payload;
54     uint8_t    crc8;
55     uint8_t    stop;
56 } telemetry_frame_t;
57
58 typedef struct
59 {
60     uint16_t    force_adc_raw;
61     uint16_t    knee_angle_adc_raw;
62     uint16_t    battery_adc_raw;

```

```
63 } adc_sample_set_t;  
64  
65 typedef struct  
66 {  
67     uint8_t    spool_angle_pct;  
68     int16_t    force_value;  
69     int16_t    knee_angle_value;  
70     int16_t    moment_value;  
71     uint16_t   battery_cV;  
72     knee_state_t state;  
73     fault_code_t fault;  
74 } system_status_t;  
75  
76 #endif
```

Listing 4.2: Shared enums, packed structs, and runtime types

Why each line exists

- The guard prevents duplicate type definitions.
- `stdint.h` gives exact-width types so packet layout does not change across toolchains.
- `stdbool.h` makes boolean state readable and explicit.
- `filter_mode_t` allows runtime selection of a filter algorithm.
- `knee_state_t` provides readable symbolic names for gait and system states.
- `fault_code_t` gives every safety error a stable identifier.
- `telemetry_payload_t` is the compact binary payload referenced throughout Chapter 3.
- `packed` prevents unwanted alignment padding so the UART payload stays exactly sized.
- `telemetry_frame_t` wraps the payload with framing and checksum fields.
- `adc_sample_set_t` groups raw ADC channel data into a semantic struct.
- `system_status_t` becomes the main shared runtime data block between tasks.

4.3 crc8.h / crc8.c

```

1 #ifndef CRC8_H
2 #define CRC8_H
3
4 #include <stdint.h>
5 #include <stdint.h>
6
7 uint8_t CRC8_Calculate(const uint8_t *data, size_t length);
8
9 #endif

```

Listing 4.3: CRC-8 checksum interface

Why each line exists

- `stdint.h` provides `size_t` for portable byte-count arguments.
- The function declaration lets any communication module call the checksum routine.

```

1 #include "crc8.h"
2
3 uint8_t CRC8_Calculate(const uint8_t *data, size_t length)
4 {
5     uint8_t crc = 0x00U;
6     size_t index;
7     uint8_t bit;
8
9     for (index = 0U; index < length; index++)
10    {
11        crc ^= data[index];
12
13        for (bit = 0U; bit < 8U; bit++)
14        {
15            if ((crc & 0x80U) != 0U)
16            {
17                crc = (uint8_t)((crc << 1U) ^ 0x07U);
18            }
19            else
20            {
21                crc <<= 1U;
22            }
23        }
24    }
25
26    return crc;
27 }

```

Listing 4.4: CRC-8 checksum implementation

Why each line exists

- The header include guarantees the implementation matches the declared prototype.
- `crc` stores the running checksum state.
- `index` walks through each byte of the input buffer.
- `bit` walks through all 8 bits of one byte.
- The outer loop processes one input byte at a time.
- `crc ^= data[index]` mixes the next byte into the current CRC state.
- The inner loop applies the chosen CRC polynomial one bit at a time.
- `0x80U` checks whether the current top bit is set before shifting.
- `0x07U` is the CRC-8 polynomial used in this package.
- The cast keeps the final value bounded to 8 bits after arithmetic promotion.
- `return crc` provides the final checksum for the caller to store in a frame.

4.4 filter.h / filter.c

```

1  #ifndef FILTER_H
2  #define FILTER_H
3
4  #include <stdint.h>
5  #include "app_types.h"
6
7  void Filter_Init(void);
8  void Filter_SetMode(filter_mode_t mode);
9  int16_t Filter_Apply(int16_t input);
10 int16_t Filter_ApplyEMA(int16_t input);
11 int16_t Filter_ApplyMovingAverage(int16_t input);
12
13 #endif

```

Listing 4.5: Filter module interface

```

1  #include "filter.h"
2
3  #define EMA_ALPHA_NUMERATOR      (1)
4  #define EMA_ALPHA_DENOMINATOR   (5)
5  #define MA_WINDOW_SIZE          (8U)
6
7  static filter_mode_t g_filter_mode = FILTER_MODE_EMA;
8  static int32_t g_ema_state = 0;
9  static int16_t g_ma_buffer[MA_WINDOW_SIZE];
10 static uint8_t g_ma_index = 0U;
11 static int32_t g_ma_sum = 0;
12
13 void Filter_Init(void)
14 {
15     uint8_t i;
16
17     g_filter_mode = FILTER_MODE_EMA;
18     g_ema_state = 0;
19     g_ma_index = 0U;
20     g_ma_sum = 0;
21
22     for (i = 0U; i < MA_WINDOW_SIZE; i++)
23     {
24         g_ma_buffer[i] = 0;
25     }
26 }
27
28 void Filter_SetMode(filter_mode_t mode)
29 {
30     g_filter_mode = mode;
31 }
32
33 int16_t Filter_Apply(int16_t input)
34 {
35     int16_t output;
36
37     switch (g_filter_mode)
38     {
39         case FILTER_MODE_MOVING_AVERAGE:
40             output = Filter_ApplyMovingAverage(input);
41             break;
42
43         case FILTER_MODE_KALMAN_PLACEHOLDER:
44             output = Filter_ApplyEMA(input);
45             break;

```

```
46
47     case FILTER_MODE_EMA:
48     default:
49         output = Filter_ApplyEMA(input);
50         break;
51     }
52
53     return output;
54 }
55
56 int16_t Filter_ApplyEMA(int16_t input)
57 {
58     g_ema_state = g_ema_state + (((int32_t)input - g_ema_state) *
59         EMA_ALPHA_NUMERATOR) / EMA_ALPHA_DENOMINATOR);
60     return (int16_t)g_ema_state;
61 }
62
63 int16_t Filter_ApplyMovingAverage(int16_t input)
64 {
65     g_ma_sum -= g_ma_buffer[g_ma_index];
66     g_ma_buffer[g_ma_index] = input;
67     g_ma_sum += input;
68
69     g_ma_index++;
70     if (g_ma_index >= MA_WINDOW_SIZE)
71     {
72         g_ma_index = 0U;
73     }
74
75     return (int16_t)(g_ma_sum / (int32_t)MA_WINDOW_SIZE);
76 }
```

Listing 4.6: Filter module implementation (EMA and moving average)

Why each line exists

- `EMA_ALPHA_NUMERATOR` and `EMA_ALPHA_DENOMINATOR` define the fixed-point EMA coefficient without using floating-point math.
- `MA_WINDOW_SIZE` sets how many samples the moving average stores.
- `g_filter_mode` stores the active algorithm so the rest of the system can use one generic function.
- `g_ema_state` is kept in `int32_t` to reduce overflow risk during intermediate EMA arithmetic.
- `g_ma_buffer` stores the last N samples for moving average mode.
- `g_ma_index` tracks the oldest sample slot to overwrite next.
- `g_ma_sum` stores the rolling sum so the algorithm is efficient.
- `Filter_Init()` resets all internal filter state.
- The clear loop ensures the moving average starts from known data.
- `Filter_SetMode()` lets Bluetooth commands or configuration code select a filter.
- `Filter_Apply()` hides algorithm choice from the caller.
- The `switch` makes mode selection explicit and readable.
- The placeholder Kalman mode deliberately falls back to EMA so the system stays functional before a full Kalman implementation exists.
- `Filter_ApplyEMA()` implements low-memory first-order IIR smoothing.
- `Filter_ApplyMovingAverage()` uses a rolling-sum technique so each new sample is cheap to process.

4.5 adc_manager.h / adc_manager.c

```

1  #ifndef ADC_MANAGER_H
2  #define ADC_MANAGER_H
3
4  #include <stdint.h>
5  #include "app_types.h"
6
7  void ADC_Manager_Init(void);
8  void ADC_Manager_StartDMA(void);
9  void ADC_Manager_Calibrate(void);
10 void ADC_Manager_ProcessBuffer(void);
11 uint16_t ADC_Manager_GetRawForce(void);
12 int16_t ADC_Manager_GetFilteredForce(void);
13 uint16_t ADC_Manager_GetBattery_cV(void);
14 void ADC_Manager_SetForceZeroOffset(uint16_t offset);
15
16 #endif

```

Listing 4.7: ADC manager interface

```

1  #include "adc_manager.h"
2  #include "config.h"
3  #include "filter.h"
4  #include "safety.h"
5
6  extern ADC_HandleTypeDef hadc1;
7
8  static uint16_t g_adc_dma_buffer[ADC_DMA_SAMPLE_COUNT];
9  static uint16_t g_force_zero_offset = ADC_FORCE_ZERO_OFFSET_DEFAULT;
10 static uint16_t g_force_raw = 0U;
11 static int16_t g_force_filtered = 0;
12 static uint16_t g_battery_cV = 0U;
13
14 static int16_t ADC_Manager_ConvertForce(uint16_t raw_adc);
15 static uint16_t ADC_Manager_ConvertBattery(uint16_t raw_adc);
16
17 void ADC_Manager_Init(void)
18 {
19     uint32_t i;
20
21     g_force_zero_offset = ADC_FORCE_ZERO_OFFSET_DEFAULT;
22     g_force_raw = 0U;
23     g_force_filtered = 0;
24     g_battery_cV = 0U;
25
26     for (i = 0U; i < ADC_DMA_SAMPLE_COUNT; i++)
27     {
28         g_adc_dma_buffer[i] = 0U;
29     }
30
31     Filter_Init();
32 }
33
34 void ADC_Manager_StartDMA(void)
35 {
36     if (HAL_ADC_Start_DMA(&hadc1, (uint32_t *)g_adc_dma_buffer,
37                          ADC_DMA_SAMPLE_COUNT) != HAL_OK)
38     {
39         Safety_SetFault(FAULT_ADC);
40     }
41 }

```

```

42 void ADC_Manager_Calibrate(void)
43 {
44     if (HAL_ADCEx_Calibration_Start(&hadc1, ADC_SINGLE_ENDED) != HAL_OK)
45     {
46         Safety_SetFault(FAULT_ADC);
47     }
48 }
49
50 void ADC_Manager_ProcessBuffer(void)
51 {
52     uint32_t sum_force = 0U;
53     uint32_t sample_count = 0U;
54     uint32_t i;
55
56     for (i = 0U; i < ADC_DMA_SAMPLE_COUNT; i++)
57     {
58         sum_force += g_adc_dma_buffer[i];
59         sample_count++;
60     }
61
62     if (sample_count == 0U)
63     {
64         Safety_SetFault(FAULT_ADC);
65         return;
66     }
67
68     g_force_raw = (uint16_t)(sum_force / sample_count);
69     g_force_filtered = Filter_Apply(ADC_Manager_ConvertForce(g_force_raw));
70     g_battery_cV = ADC_Manager_ConvertBattery(g_force_raw);
71
72     if ((g_force_filtered > FORCE_LIMIT_ABS) || (g_force_filtered < -
73         FORCE_LIMIT_ABS))
74     {
75         Safety_SetFault(FAULT_SENSOR_RANGE);
76     }
77
78     if ((g_battery_cV > BATTERY_MAX_CV) || (g_battery_cV < BATTERY_MIN_CV))
79     {
80         Safety_SetFault(FAULT_BATTERY);
81     }
82
83     uint16_t ADC_Manager_GetRawForce(void)
84     {
85         return g_force_raw;
86     }
87
88     int16_t ADC_Manager_GetFilteredForce(void)
89     {
90         return g_force_filtered;
91     }
92
93     uint16_t ADC_Manager_GetBattery_cV(void)
94     {
95         return g_battery_cV;
96     }
97
98     void ADC_Manager_SetForceZeroOffset(uint16_t offset)
99     {
100         g_force_zero_offset = offset;
101     }
102
103     static int16_t ADC_Manager_ConvertForce(uint16_t raw_adc)
104     {
105         int32_t signed_delta;

```

```
106     int32_t scaled_force;
107
108     signed_delta = (int32_t)raw_adc - (int32_t)g_force_zero_offset;
109     scaled_force = (signed_delta * ADC_FORCE_SCALE_NUM) / ADC_FORCE_SCALE_DEN;
110
111     return (int16_t)scaled_force;
112 }
113
114 static uint16_t ADC_Manager_ConvertBattery(uint16_t raw_adc)
115 {
116     uint32_t scaled;
117
118     scaled = ((uint32_t)raw_adc * BATTERY_SCALE_CV_NUM) / BATTERY_SCALE_CV_DEN;
119     return (uint16_t)scaled;
120 }
```

Listing 4.8: ADC manager implementation (DMA acquisition, calibration, conversion)

Why each line exists

- `extern ADC_HandleTypeDef hadc1;` imports the HAL ADC handle defined in the board initialization layer.
- `g_adc_dma_buffer` is the DMA destination array for raw ADC data.
- `g_force_zero_offset` stores the current force-channel zero calibration point.
- `g_force_raw`, `g_force_filtered`, and `g_battery_cV` keep the latest processed values ready for other modules.
- Static helper prototypes keep internal conversion functions private to this module.
- `ADC_Manager_Init()` resets all state and clears the DMA buffer.
- The buffer-clear loop ensures that stale RAM is not interpreted as live ADC data.
- `Filter_Init()` is called here because filtering is tightly coupled to ADC processing.
- `ADC_Manager_StartDMA()` starts continuous ADC-to-memory transfer using HAL DMA support.
- If DMA start fails, the system latches an ADC fault instead of continuing with invalid assumptions.
- `ADC_Manager_Calibrate()` explicitly runs ADC calibration because calibration improves raw measurement correctness on STM32 ADC paths.
- `ADC_Manager_ProcessBuffer()` averages the DMA window, which reduces high-frequency noise before the configured filter stage.
- `sample_count` is counted explicitly to make the average calculation clear and defensive.
- The zero-sample guard is defensive coding for abnormal states.
- `ADC_Manager_ConvertForce()` subtracts the calibrated zero point so the force channel becomes bipolar.
- `Filter_Apply()` lets the active algorithm be swapped without changing caller logic.
- `ADC_Manager_ConvertBattery()` converts raw ADC to centivolts using integers only.
- The force and battery limit checks turn invalid measurements into safety faults immediately.
- Getter functions provide controlled read access to module state without exposing raw globals.
- `ADC_Manager_SetForceZeroOffset()` supports runtime zeroing or calibration storage restore.

4.6 safety.h

```
1 #ifndef SAFETY_H
2 #define SAFETY_H
3
4 #include <stdbool.h>
5 #include "app_types.h"
6
7 void Safety_Init(void);
8 void Safety_SetFault(fault_code_t fault);
9 void Safety_ClearFaults(void);
10 bool Safety_HasFault(void);
11 fault_code_t Safety_GetFault(void);
12 void Safety_ApplySafeState(void);
13
14 #endif
```

Listing 4.9: Safety supervisor interface

The original blueprint document was truncated at this point – the `safety.c` implementation, and every file after it in Table 2.1 (`state_machine`, `motor_control`, `uart_dma`, `bluetooth`, `log_flash`, `freertos_app`, and `main.c`), were referenced in the architecture and project tree but never supplied with actual source. This interface is included because it was fully specified, but its implementation should be written and reviewed before this firmware is built for a physical device.

CHAPTER 5

Runtime Data and Memory Footprint

This chapter consolidates the in-memory footprint of the structures and buffers defined across Chapter 4, which is useful when budgeting SRAM against the STM32U585AI's 786 KB total.

5.1 telemetry_payload_t Layout

Because `telemetry_payload_t` (Listing 4.2) is declared `__attribute__((packed))`, no padding is inserted between members and the struct occupies exactly the sum of its field widths.

Table 5.1: Byte layout of `telemetry_payload_t`

Offset	Field	Type	Width
0	spool_angle_pct	uint8_t	1 byte
1	force_raw	int16_t	2 bytes
3	knee_angle_raw	int16_t	2 bytes
5	moment_raw	int16_t	2 bytes
7	state_id	uint8_t	1 byte
8	battery_cV	uint16_t	2 bytes
Total payload size			10 bytes

Adding the `telemetry_frame_t` framing overhead (start, length, type, CRC8, stop – five additional bytes) gives a complete telemetry frame of **15 bytes**, comfortably inside both UART DMA RX buffers (128 bytes each, Listing 4.1) and well within a single Bluetooth low-energy notification payload.

5.2 Static RAM Allocation Summary

Table 5.2 lists every statically-allocated buffer or state variable defined in the modules typeset in Chapter 4. This is not the full firmware RAM budget – FreeRTOS task stacks, heap, and the unimplemented modules from Table 2.1 are not yet accounted for – but it is a useful running total for the acquisition and filtering path.

Independent of SRAM, `FLASH_LOG_REGION_SIZE` (Listing 4.1) reserves `FLASH_LOG_PAGE_SIZE × FLASH_LOG_REGION_PAGES = 8192 × 4 = 32768` bytes (32 KB) of Bank 2 Flash starting at `0x08100000`, for persistent fault and calibration records. This is non-volatile and does not count against the 786 KB SRAM budget.

5.3 Fault Code Reference

Table 5.3 reproduces the `fault_code_t` enumeration from `app_types.h` (Listing 4.2) alongside the module currently responsible for raising each fault, based on the code typeset in Chapter 4.

Table 5.2: Static RAM usage of completed modules

Module	Symbol	Size
adc_manager.c	g_adc_dma_buffer[32] (uint16_t)	64 bytes
filter.c	g_ma_buffer[8] (int16_t)	16 bytes
adc_manager.c	g_force_zero_offset (uint16_t)	2 bytes
adc_manager.c	g_force_raw (uint16_t)	2 bytes
adc_manager.c	g_force_filtered (int16_t)	2 bytes
adc_manager.c	g_battery_cV (uint16_t)	2 bytes
filter.c	g_ema_state (int32_t)	4 bytes
filter.c	g_ma_sum (int32_t)	4 bytes
filter.c	g_ma_index (uint8_t)	1 byte
filter.c	g_filter_mode (enum)	4 bytes
Subtotal		101 bytes

Table 5.3: Fault codes and raising modules

Fault Code	Raised By	Condition
FAULT_NONE	–	No active fault.
FAULT_ADC	adc_manager.c	DMA start failure, calibration failure, or zero valid samples in the acquisition window.
FAULT_SENSOR_RANGE	adc_manager.c	Filtered force magnitude exceeds FORCE_LIMIT_ABS.
FAULT_COMM_TIMEOUT	pending (uart_dma.c)	Host link exceeds COMMAND_TIMEOUT_MS without a valid frame.
FAULT_MOTOR	pending (motor_control.c)	Reserved for motor/actuator faults.
FAULT_BATTERY	adc_manager.c	Battery voltage outside BATTERY_MIN_CV–BATTERY_MAX_CV.
FAULT_WATCHDOG	pending (freertos_app.c)	Reserved for RTOS task watchdog supervision.
FAULT_FLASH	pending (log_flash.c)	Reserved for Flash write/erase failures.

5.4 Closing Notes

Chapter 4 typesets every module that was completely specified in the original blueprint, and Table 2.1 is an honest accounting of what remains to be written: the safety supervisor’s logic, the gait/system state machine, PWM motor control, both UART DMA framing layers, Bluetooth command handling, the Flash-resident logger, FreeRTOS task orchestration, and `main.c` bring-up. Given that this firmware ultimately drives a hydraulic valve in a load-bearing prosthetic knee, the recommended next step is to complete and independently review those modules – particularly `safety.c` and `motor_control.c` – before any of this code is flashed to hardware that a person will rely on.